

■

EGA - Carga Electrónica Programable

INDICE

1. ABSTRACT	2
PALABRAS CLAVE:.....	2
2. FUNDAMENTACION.....	2
3. FUNDAMENTACIÓN TEÓRICA DEL SOFTWARE.....	2
4. FUNDAMENTACIÓN TEÓRICA DEL HARDWARE.....	2
5. OBJETIVOS.....	3
6. ACERCA DEL HARDWARE.....	3
6.1. DS3231 (RTC)	3
6.2. DS3231 (EEPROM)	3
6.3. DAC MCP4725	3
6.4. TFT DISPLAY 2,4"	3
6.5. ESP32 S3	4
8. ESQUEMATICO	4
9. TEST POINT	4
10. BIBLIOGRAFIA.....	4
ANEXO I – DIAGRAMA EN BLOQUES	7
ANEXO II – DIAGRAMA DE TAREAS	8
ANEXO III – ESQUEMATICO	9
ANEXO IV – VISTA 3D PLACA	11
ANEXO V – VISTA REAL DE LA PLACA.....	12
ANEXO VI – VISTA DEL EQUIPO.....	13
ANEXO VII – TEST POINT	14
ANEXO VIII – CALIBRACION	14
ANEXO IV – GUIA RAPIDA DE CODIGO.....	15
ANEXO IIV – MANUAL DE USO	17

EGA - Carga Electrónica Programable

Celada, Matias

matiasicelada@gmail.com

Martinez, Ignacio

matiasicelada@gmail.com

1. ABSTRACT.

Este proyecto presenta el desarrollo de una carga electrónica programable para laboratorio, diseñada como un instrumento versátil para la caracterización y ensayo de sistemas de energía, tales como fuentes de alimentación y bancos de baterías. El principio de funcionamiento se basa en la regulación dinámica de corriente a través de un transistor de potencia operado en su región activa. El control analógico del dispositivo se gestiona mediante un microcontrolador, el cual permite programar perfiles de carga específicos y variar el consumo de corriente de forma precisa según los requerimientos del ensayo.

PALABRAS CLAVE:

Laboratorio, Carga, Electrónica, Potencia, Fuente, Programable.

2. FUNDAMENTACION

Cuando se realiza la construcción de una fuente de laboratorio, es fundamental realizarle distintas pruebas para poder elaborar su hoja de especificaciones. Entre las evaluaciones más comunes se encuentran la regulación por carga y como responde dinámicamente la fuente frente a las variaciones de consumo de corriente.

Si se desarrollan varias fuentes con diferentes tensiones y corrientes nominales, realizar el ensayo sería engorroso ya que debería haber una carga para cada fuente en particular, de manera de verificar su funcionamiento en las condiciones máximas especificadas y confirmar que regulan correctamente. Esto implica disponer de grandes bancos de resistencias.

Con el instrumento propuesto, estos inconvenientes se solucionarían: se podrán programar diferentes valores de carga, y la prueba dinámica podría realizarse automáticamente, variando en diferentes tensiones y corrientes, sin intervención del operador.

3. FUNDAMENTACIÓN TEÓRICA DEL SOFTWARE

Para el desarrollo del firmware se seleccionó el sistema operativo de tiempo real FreeRTOS, debido a su capacidad para estructurar el flujo del programa mediante una arquitectura orientada a tareas concurrentes con prioridades determinísticas. Esta planificación asegura un comportamiento de tiempo real estricto (hard real-time), garantizando que las sub tareas críticas —como la adquisición y digitalización de variables eléctricas— se ejecuten de manera oportuna y sin pérdida de datos.

4. FUNDAMENTACIÓN TEÓRICA DEL HARDWARE

Para realizar las mediciones de Tensión y Corriente del equipo se utilizarán dos ADC, uno para leer la Tensión de alimentación de la fuente de prueba, y otro ADC que va a leer la caída de tensión sobre la resistencia **R SHUNT**, el cual mediante la realización de cálculos en el micro controlador, expresará el nivel de corriente.

Los ADC utilizados son los internos del microcontrolador:

- **Canales:** ADC0 y ADC1 de 12 bits
- **Resolución:** 12 bits → 0–4095 pasos.
- **Rango de tensión:** 0 V a 3,3 V
- **Velocidad de muestreo:** 1000 muestras/seg.

Para realizar el control del mosfet, se decidió utilizar un DAC (MCP4725), ya que, controlarlo con un filtro limitaba mucho la velocidad de muestreo, debido a la resolución del PWM.

Las características son las siguientes:

- **Tipo:** DAC de 1 canal, resolución de 12 bits (0–4095 pasos).
- **Interfaz:** Comunicación I2C
- **Rango de salida:** de 0 V a 5V
- **Voltaje de alimentación:** 2,7 V a 5,5 V
- **Memoria EEPROM interna:** guarda el último valor de salida y lo recupera al encender.
- **Tiempo de establecimiento** (settling time): ~6 μ s típico.
- **Dirección I²C configurable:** mediante conexión del pin A0 (0x60 o 0x61).

5. OBJETIVOS

- Realizar un pre-seteo de N valores de resistencias
- Medir en tiempo real la tensión de entrada, corriente de la carga y valor de resistencia de setpoint seleccionado.
- Almacenar las alarmas en una memoria EEPROM
- Ajustar la corriente de carga mediante un control PI.
- Detectar valores de corriente y tensión que superen los umbrales preseleccionados por el usuario y apagar el equipo por seguridad.

6. ACERCA DEL HARDWARE

La electrónica y los módulos a utilizar se detallarán a continuación:

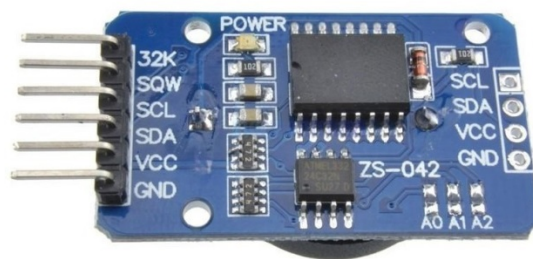
6.1. DS3231 (RTC)

- **Tensión de alimentación:** 5V.
- **Comunicación:** interfaz I2C (dirección 0x68).
- **Precisión:** ± 2 ppm ($\approx \pm 1$ min/año) entre 0 °C y 40 °C.
- Oscilador compensado por temperatura (TCXO) integrado.
- **Formato de hora:** 24 h o 12 h con AM/PM.
- Batería de respaldo para mantener la hora sin alimentación.
- **Rango de temperatura:** -40 °C a +85 °C.

6.2. DS3231 (EEPROM)

- **Tensión de alimentación:** 5V.
- **Capacidad:** 32 Kb (4 KB).
- **Comunicación:** interfaz I2C (dirección 0x57).
- **Tiempo de escritura:** 5 ms por página (32 bytes).
- Conserva los datos sin alimentación (memoria no volátil).

- **Vida útil:** 1 millón de ciclos de escritura por celda.
- **Velocidad de comunicación:** 100 kHz.



6.3. DAC MCP4725



- **Tensión de alimentación:** 5V
- **Comunicación:** interfaz I2C (dirección por defecto 0x60).
- **Resolución:** 12 bits (4096 niveles).
- **Ventada y Vsalida:** 0-5V
- Salida analógica de voltaje proporcional al valor digital.
- Memoria EEPROM interna para almacenar el último valor configurado.
- **Velocidad de actualización:** hasta 3,4 Mbps en modo I2C rápido.
- Bajo consumo de energía en modo standby.
- **Rango de temperatura de operación:** -40 °C a +125 °C.

6.4. TFT DISPLAY 2,4"



- **Tiempo de establecimiento** (settling time): ~6 μ s típico.
- **Dirección I²C configurable**: mediante conexión del pin A0 (0x60 o 0x61).
- **Tensión de alimentación**: 3.3V a 5V (los módulos suelen integrar su propio regulador de voltaje).
- **Comunicación**: Interfaz SPI de 4 hilos (reduce la cantidad de pines necesarios del microcontrolador).
- **Resolución**: 240 x 320 píxeles (QVGA).
- **Controlador (Driver IC)**: ILI9341 (o equivalentes como el ST7789).
- **Profundidad de color**: 16 bits (65.536 colores) o 18 bits (262.144 colores).
- **Nivel lógico de señales**: 3.3V (requiere conversores de nivel si el microcontrolador opera a 5V).
- **Hardware adicional integrado**: Zócalo para tarjeta MicroSD (utiliza el mismo bus SPI).
- **Interfaz táctil**: Usualmente disponible con panel táctil de tipo resistivo (mediante el controlador XPT2046).
- **Rango de temperatura de operación**: -20 °C a +70 °C.

6.5. ESP32 S3



- **Tensión de alimentación**: 3.0V a 3.6V (el chip opera nativamente a 3.3V).
- **Núcleo del procesador**: Microprocesador Xtensa® de 32 bits LX7 de doble núcleo, con una frecuencia de reloj de hasta 240 MHz.

- **Conectividad inalámbrica**: Wi-Fi de 2.4 GHz (802.11 b/g/n) y Bluetooth® 5 (LE) con soporte para conexiones de largo alcance.
- **Memoria interna**: 384 KB de ROM, 512 KB de SRAM y 16 KB de SRAM en el RTC (soporta expansión de memoria Flash y PSRAM externas por bus SPI/Octal SPI).
- **Pines de entrada/salida (GPIO)**: Hasta 45 pines programables.
- **Interfaces de comunicación**: Múltiples buses que incluyen SPI, I2C, I2S, UART, TWAI® (compatible con CAN), USB 1.1 OTG y SD/SDIO/MMC.
- **Periféricos analógicos**: ADC de 12 bits (hasta 20 canales) y sensor de temperatura interno integrado.
- **Características especiales**: Aceleración por hardware para Inteligencia Artificial (IA) mediante instrucciones vectoriales para procesamiento de señales y redes neuronales.
- **Bajo consumo**: Múltiples modos de energía (Deep-sleep, Light-sleep) apoyados por el coprocesador de ultra bajo consumo (ULP).
- **Rango de temperatura de operación**: -40 °C a +85 °C (hasta +105 °C dependiendo de la variante específica del chip).

7. ESQUEMATICO

El equipo se diseñó en una única placa, en la cual se encuentran todos los módulos y componentes necesarios para su funcionamiento.

La alimentación deberá ser externa mediante un transformador DC o una fuente de tensión continua de 12V.

Tanto los esquemáticos como los diseños fueron hechos en Kicad 9.0.

Para más detalles, consultar el "Anexo III"

8. TEST POINT

El hardware diseñado incorpora **TEST POINTS** que facilitara la realización de las mediciones correspondientes en cada uno de ellos.

Estos puntos se identifican con un nombre descriptivo precedido por las siglas "TP", lo que permite ubicarlos con mayor rapidez.

Para más detalles, consultar el "Anexo III"

■

9. ACERCA DEL PROGRAMA

El sistema permite disipar energía de una fuente bajo prueba en modos de Corriente Constante (CC) y Resistencia Constante (CR), implementando un lazo de control digital cerrado (PID) y protecciones contra sobretensión (OVP) y sobrecorriente (OCP).

Para garantizar determinismo temporal y modularidad, el software se divide en cinco tareas concurrentes sincronizadas mediante colas y exclusión mutua (Mutex):

Task_ADC (Adquisición y Filtrado): Muestra periódicamente (cada 50 ms / 20 Hz) los canales de tensión y corriente del ADC interno, aplicando un filtro digital de promedio móvil exponencial (EMA) y tablas de calibración para eliminar ruido y no linealidades.

Task_PID (Lazo de Control): Se sincroniza con cada nueva muestra del ADC para calcular el error entre la consigna y la corriente real. Ejecuta el algoritmo PID con límites anti-windup y calcula la señal de control en un rango de 12 bits (0 a 4095).

Task_DAC (Actuación Analógica): Recibe la salida digital del PID y comanda mediante bus I2C (en Fast-Mode y protegido por un Mutex) al convertidor digital-analógico externo MCP4725, el cual polariza la compuerta del transistor de potencia (MOSFET).

Task_SysMan (Gestor del Sistema y Seguridad): Actúa como supervisor central. Procesa los cambios de modo y setpoints ingresados por el usuario, evalúa permanentemente las condiciones de sobretensión (OVP) y sobrecorriente (OCP), y activa el corte de emergencia (force stop) junto con los LEDs indicadores ante cualquier fallo.

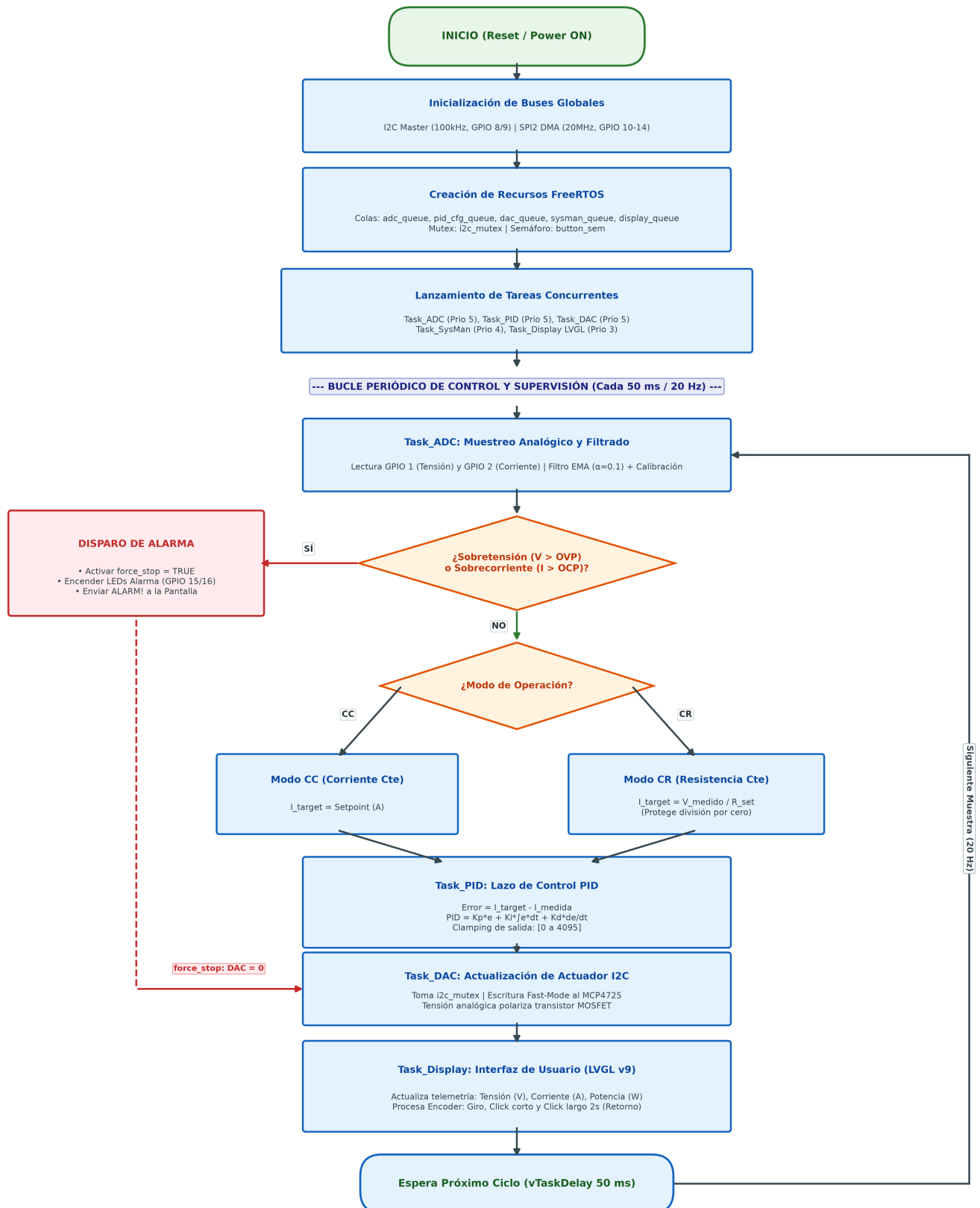
Task_Display (Interfaz de Usuario - HMI): Controla el display TFT (ILI9341 por bus SPI) utilizando la librería gráfica LVGL v9. Muestra la telemetría en tiempo real (V, A, W) y procesa las entradas del encoder rotativo, reconociendo navegación, selección y retorno al menú principal mediante pulsación larga.

■

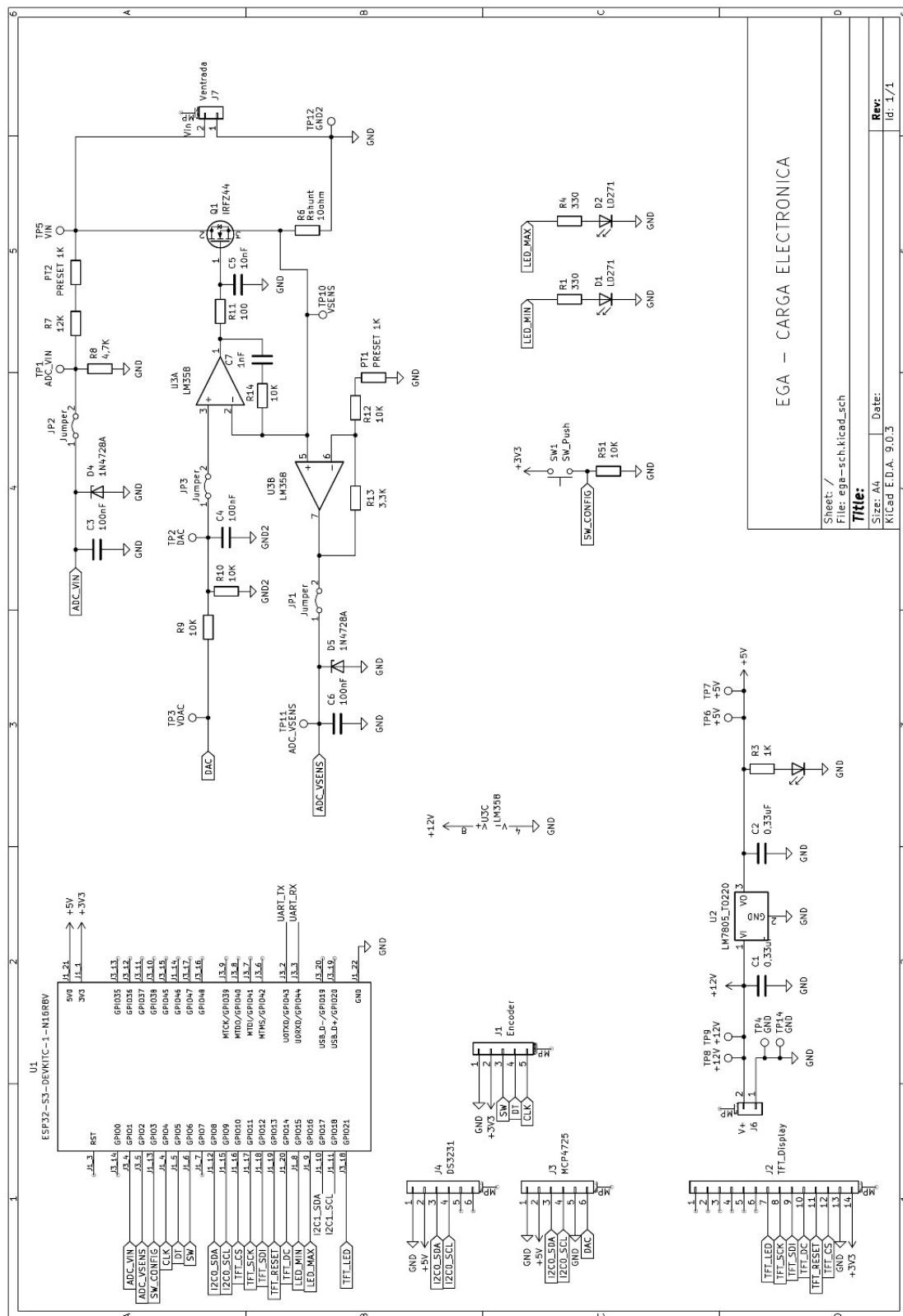
10. BIBLIOGRAFIA:

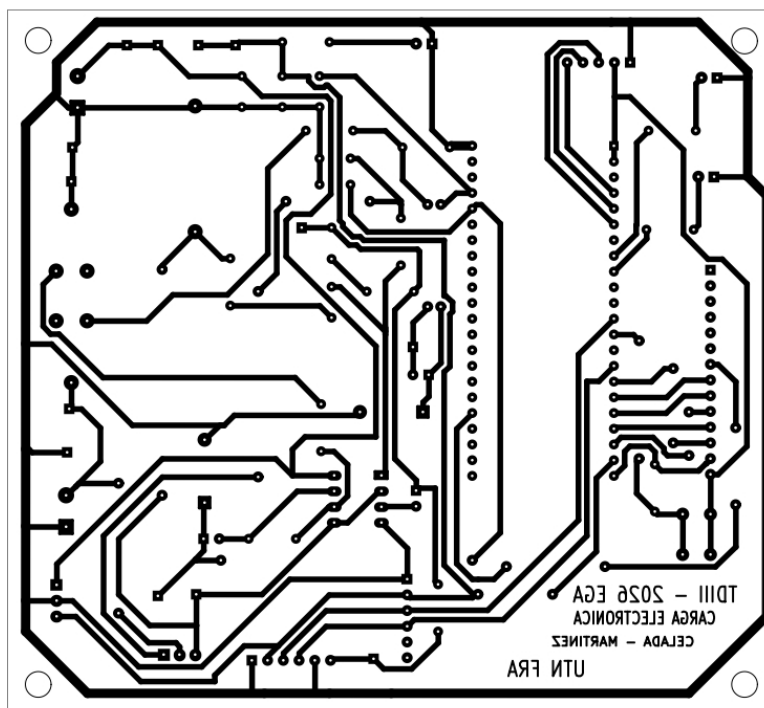
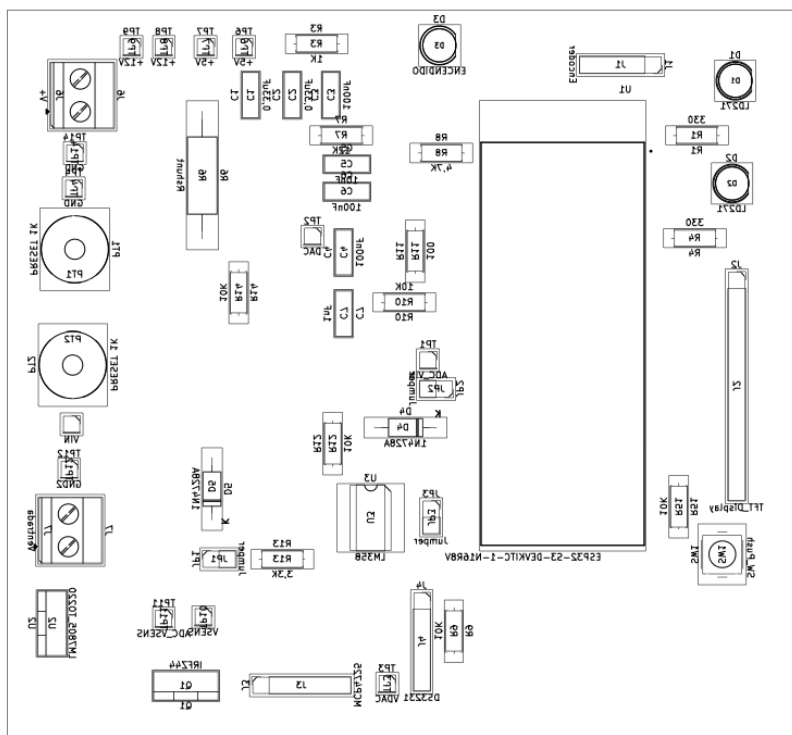
- [1] Espressif Systems, “ESP-IDF Programming Guide & API Reference (v5.x)”: Documentación oficial y librerías para periféricos (ADC, I2C, SPI, PCNT, ESP-LCD), temporizadores y FreeRTOS. [En línea: <https://docs.espressif.com/projects/esp-idf>].
- [2] Espressif Systems, “ESP32-S3 Series Datasheet & Technical Reference Manual”, Versión 1.6, 2023.
- [3] Maxim Integrated / Analog Devices, “DS3231: Extremely Accurate I²C-Integrated RTC/TCXO/Crystal Datasheet”, Rev. 10, 2015. (Módulo de reloj de tiempo real RTC para sincronización horaria del sistema).
- [4] Microchip Technology Inc., “MCP4725: 12-Bit Digital-to-Analog Converter with EEPROM Memory in SOT-23-6 Datasheet”, Doc. DS22039D, 2009. (Conversor DAC para el control del MOSFET).
- [5] Ilitek Corp., “ILI9341: a-Si TFT LCD Single Chip Driver 240RGBx320 Resolution Datasheet”, Versión 1.11, 2011. (Controlador del display gráfico).
- [6] LVGL LLC, “LVGL - Light and Versatile Graphics Library Documentation (v9.x)”, Guía de puertos, widgets y eventos. [En línea: <https://docs.lvgl.io>].
- [7] SquareLine Studio, “SquareLine Studio User Manual & Component Exporting for Embedded Systems”, 2023.

ANEXO II – DIAGRAMA DE TAREAS



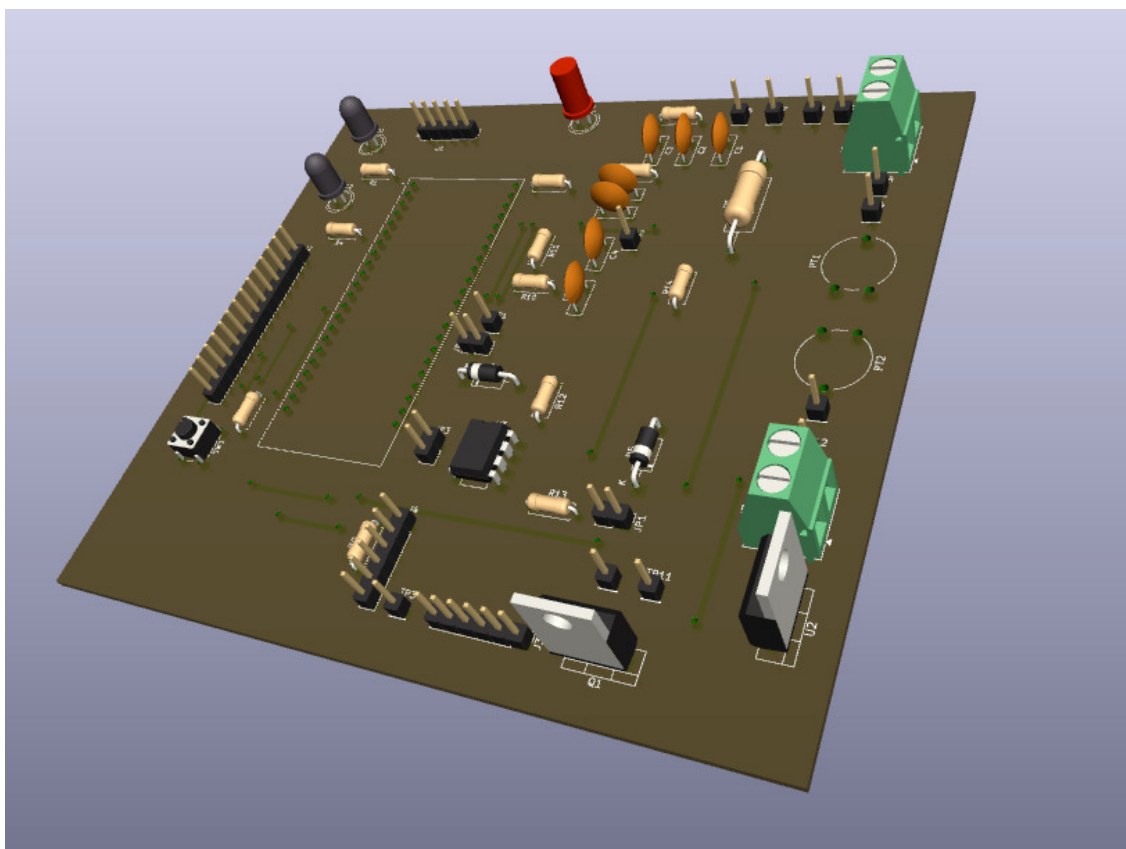
ANEXO III – ESQUEMATICO



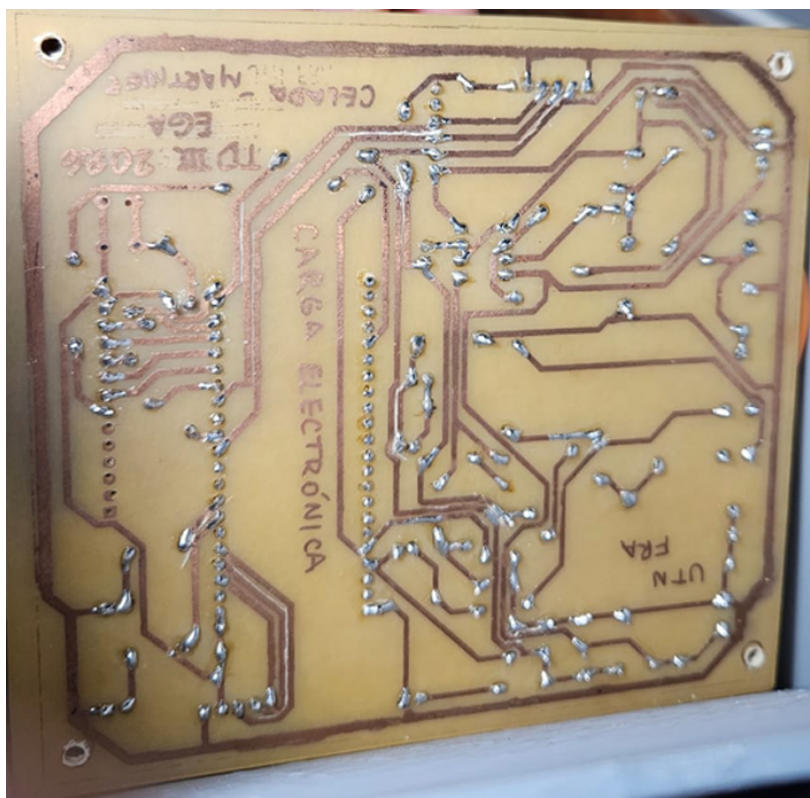
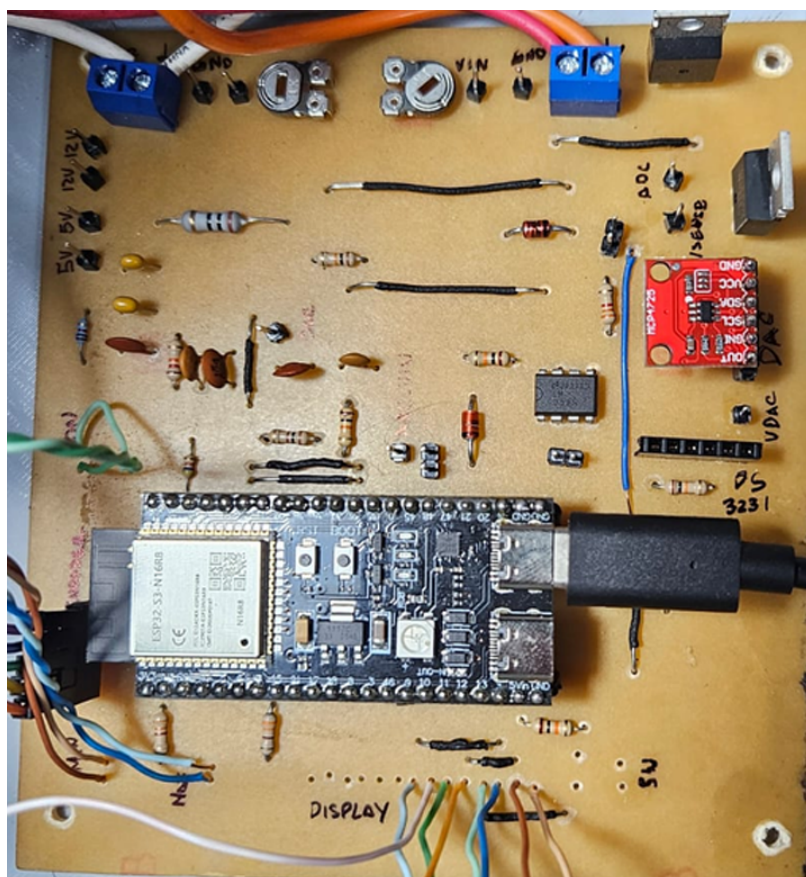


■

ANEXO IV – VISTA 3D PLACA



ANEXO V – VISTA REAL DE LA PLACA



ANEXO VI – VISTA DEL EQUIPO



ANEXO VII – TEST POINT

Test Point	Nombre	Descripción	Valor Estimado
TP1	ADC_VIN	Medición de tensión proporcional que ingresa al ADC para realizar la medición de tensión	0 - 3,3V
TP2	DAC	Señal DAC luego del divisor resistivo	0 - 2,5V
TP3	VDAC	Señal DAC previa al divisor resistivo	0 - 5V
TP4	GND	Tierra	0V
TP5	VIN	Tensión de alimentación de fuente de prueba	0 - 12V
TP6	+5V	Rail de +5V para alimentación	5V
TP7	+5V	Rail de +5V para alimentación	5V
TP8	+12V	Rail de +12V para alimentación LM7805	12V
TP9	+12V	Rail de +12V para alimentación LM7805	12V
TP10	Vshunt	Tensión de sensado proporcional a la corriente que circula por Rshunt	0 - 2,5V
TP11	ADC_SENSE	Medición de tensión proporcional que ingresa al ADC para realizar la medición de corriente sobre Rshunt	0 - 3,3V
TP12	GND	Tierra	0V
TP14	GND	Tierra	0V

ANEXO VIII – CALIBRACION

Tensión Real (V_{in} V_{in})	Tensión GPIO 1	Corriente Real (I_{in} I_{in})	Tensión GPIO 2
0 V	0,06 V (60 mV)	0,00 A	0,08 V (80 mV)
1 V	0,36 V (360 mV)	0,05 A	0,61 V (610 mV)
2 V	0,61 V (610 mV)	0,10 A	1,08 V (1080 mV)
3 V	0,85 V (850 mV)	0,15 A	1,62 V (1620 mV)
5 V	1,17 V (1170 mV)	0,20 A	2,07 V (2070 mV)
7 V	1,36 V (1360 mV)	—	—
10 V	1,51 V (1510 mV)	—	—

Nota: Para la calibración se utilizó una Rshunt de 10 Ω

ANEXO IV – GUIA RAPIDA DE CODIGO

1. Módulos y Responsabilidades

- main.c: Inicia buses (I2C/SPI), crea colas/mutexes y lanza las 5 tareas de FreeRTOS.
- adc_sensor: Lee VV e I (GPIO 1 y 2) cada 50 ms, aplica filtro EMA y calibra por tabla.
- pid_control: Calcula el lazo cerrado ($K_p=150, K_i=45, K_d=5$) para regular corriente en CC y CR.
- mcp4725: Escribe el valor digital (0–4095) en el DAC por I2C para controlar el MOSFET.
- system_manager: Gestiona modos (CC/CR), setpoints y dispara alarmas OVP/OCP cortando la carga.
- encoder: Lee giro con módulo PCNT y distingue click corto (selección) de click largo 2 s (volver al menú).
- display_tft: Controla el panel ILI9341 por SPI con LVGL v9 (sliders, menús y telemetría).
- pin_setting / communication_setting: Centralizan el pinout y la configuración de buses.

2. Colas de Comunicación (IPC)

- adc_queue: Pasa lecturas de ADC →→ PID (50 ms).
- pid_cfg_queue: Envía modo y setpoint de SysMan →→ PID.
- dac_queue: Envía el valor calculado de PID →→ DAC.
- sysman_queue: Envía eventos de UI/ADC →→ SysMan (alarmas y cambios).
- display_queue: Envía telemetría de ADC/SysMan →→ Display.
- i2c_mutex: Protege el bus I2C entre el DAC y la EEPROM.

3. Mapeo de Pines (Pinout Clave)

- ADC: GPIO 1 (Tensión) | GPIO 2 (Corriente)
- Encoder: GPIO 4 (A) | GPIO 5 (B) | GPIO 6 (Pulsador SW)
- I2C (DAC / EEPROM): GPIO 8 (SDA) | GPIO 9 (SCL)
- Display SPI: MOSI: 12 | SCLK: 11 | CS: 10 | DC: 14 | RST: 13
- LEDs Alarma: GPIO 15 (OCP) | GPIO 16 (OVP)

4. Lista de Archivos

- main/main.c: Punto de entrada (app_main); inicializa buses, crea colas/mutexes de FreeRTOS y lanza las 5 tareas principales.
- pin_setting.h: Define el mapa global de pines GPIO del ESP32-S3 (ADC, encoder y LEDs de alarma).
- pin_setting.c: Archivo de inicialización auxiliar requerido por la arquitectura de componentes de ESP-IDF.
- communication_setting.h: Declara prototipos y constantes de configuración de los buses I2C y SPI.
- communication_setting.c: Configura e inicializa el bus I2C maestro (100 kHz) y el bus SPI2 con DMA (20 MHz).
- adc_sensor.h: Declara la estructura sensor_data_t (tensión y corriente) y la tarea del ADC.
- adc_sensor.c: Tarea de muestreo a 20 Hz, filtro digital EMA ($\alpha=0.1$) y tablas de calibración por interpolación lineal.
- pid_control.h: Declara el prototipo de la tarea de control task_pid_compute.
- pid_control.c: Tarea del lazo cerrado PID (modos CC y CR), cálculo de error y limitación anti-windup (0–4095).
- mcp4725.h: Declara el prototipo de la tarea task_dac_update.
- mcp4725.c: Tarea que escribe por I2C en el DAC MCP4725 (usando i2c_mutex) para polarizar el transistor de potencia (MOSFET).
- system_manager.h: Define los modos (CC/CR/CV), alarmas y estructuras de datos (sysman_msg_t y pid_config_t).

■

- system_manager.c: Tarea supervisora; procesa órdenes de UI, vigila límites de seguridad (OVP/OCP) y activa el paro de emergencia.
- encoder.h: Declara la inicialización del encoder y el callback de pulsación larga.
- encoder.c: Driver de pulsos PCNT por hardware y lógica del pulsador SW (click corto para seleccionar y click largo de 2 s para volver al menú).
- display_tft.h: Declara la estructura de mensajes para la interfaz ui_update_t y la tarea de pantalla.
- display_tft.c: Tarea HMI; inicializa el display ILI9341 con LVGL v9, vincula controles/sliders y refresca la telemetría en tiempo real.
- eeeprom.h / eeeprom.c: Driver de bajo nivel para lectura y escritura de bytes en memoria no volátil 24C32 por I2C.
- ui/ui.c (y archivos ui *.c): Código generado por SquareLine Studio que define el diseño gráfico, widgets y estilos de cada pantalla.

ANEXO IIV – MANUAL DE USO

1. **Conectar la fuente a +12V (o mayor)**
2. **Encender con el botón**
 1. Encenderá el programa
 2. Una vez inicie, se deberá ingresar a ALARMAS
3. **Setear las alarmas**
 1. Se deberá setear primero la alarma de voltaje deseada, luego presionar click
 2. Ahora se deberá setear la alarma de corriente deseada
 3. Una vez seteado, se deberá ingresar a CONFIGURACION
4. **Seleccionar modo de uso**
 - a. Dependiendo el modo de uso seleccionado, presionar clic
 - b. A continuación, deberá setear el valor deseado, luego presionar clic
 - c. Automáticamente el programa pasara a la pantalla runing.
5. **Conectar la carga**
 - a. El equipo funcionará en modo continuo, leyendo y mostrando en la pantalla los datos actuales, el modo en que se está utilizando, y habrá dos testigos por si se activa alguna alarma por sobre voltaje o sobre corriente

DURANTE MODO CONTINUO:

1. En caso de querer parar la carga, se podrá presionar el botón en el gabinete, que apagará la carga, pero mantendrá el programa funcionando.
2. Si se mantiene apretado el botón del encoder, se volverá al menú de inicio, inhabilitando el programa hasta nuevamente entrar a INICIO y presionando el botón para encender la carga.